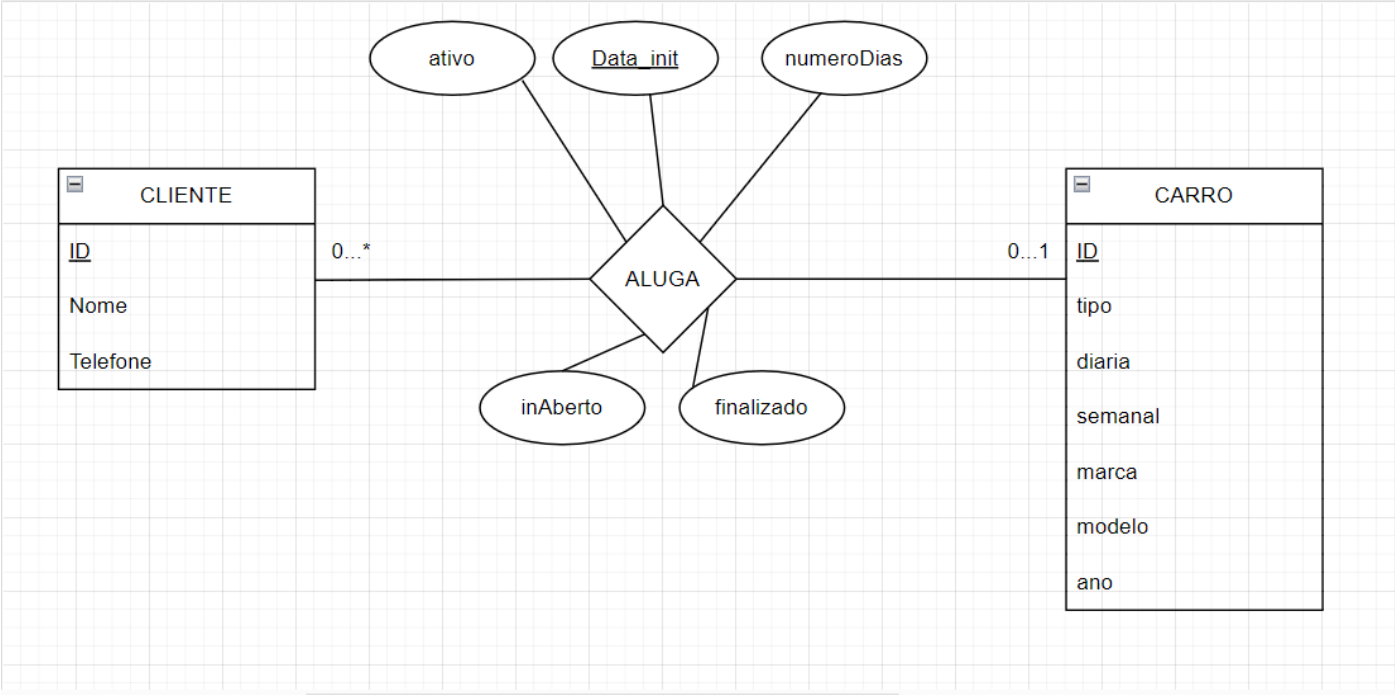
Trabalho - LAB.Banco de Dados

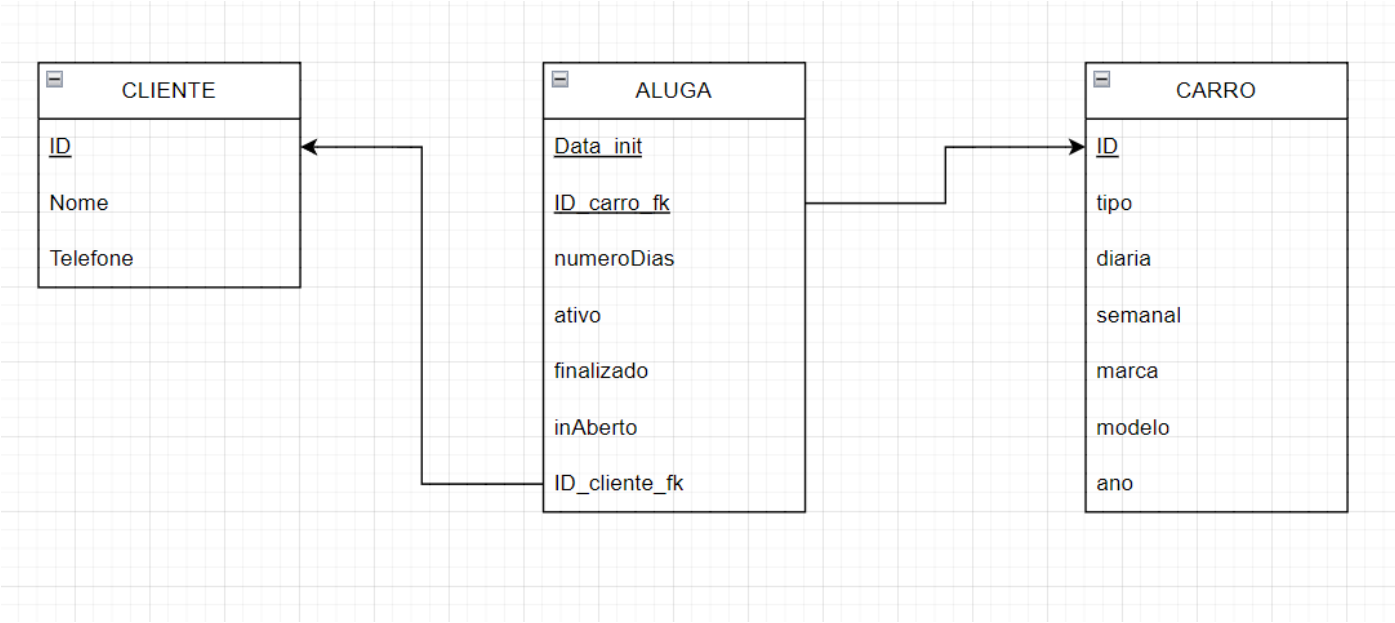
RGA: 201911722017
Nome: Matheus Rocha Rodrigues

Criação das Tabelas e Escolha das Restrições:

Criamos três tabelas, uma para identificar o carro e outra para o cliente, então criamos uma relacionamento que será convertido em uma terceira tabela armazenando os atributos de agendamento como data, númerosDeDias entre outros.



CLIENTE feito conforme pedido.
CARRO possui *ID, tipo ,marca, modelo e ano*. Além disso, adicionamos “diária e semanal”. Diária e semanal foi feita para cumprir o requisito de poder cadastrar e atualizar o valores. Sendo *diária* o valor do dia, e *semanal* o valor extra ou menos, dependendo de como você quer alterar, vai ser escolhido antes de fazer um aluguel. Tudo vai ser explicado depois nas interfaces.
ALUGA vai receber *Data_init* para saber a data de início do aluguel/agendamento. Além disso, vai dizer se está *ATIVO, FINALIZADO e EM ABERTO*, além de contar os dias de aluguel que vão ser calculados depois via python.



Já no relacional **ALUGA** vira tabela e recebe ID do CLIENTE como FK, e ID CARRO como FK, sendo que a *super key* aqui é **Data_init** e **ID_carro_fk**, com isso já o suficiente para identificar os alugueis, sendo que um cliente pode alugar o mesmo carro, mas n pode alugar no mesmo dia ou o mesmo carro.

CÓDIGO NO MYSQL:

Aqui é o código de criação das tabelas seguindo tudo o que tinha falado.
ID de **CLIENTE** e **CARRO** com auto_increment. Telefone usando BIGINT para ter a opção de maiores números.
DIÁRIA e SEMANAL da tabela **CARRO** como numeric para poder usar números com vírgula.
Em **ALUGA** usamos boolean em ATIVO, FINALIZADO e INABERTO para representação dos status do aluguel e carro.

```
create database Locadora;

use locadora;

create table cliente (
     id int auto_increment,
    nome varchar(50),
    telefone bigint,
    primary key (id)
);
create table carro (
     id int auto_increment,
```

```
     tipo varchar(30),
    diaria numeric(10,2),
     semanal numeric(10,2),
    marca varchar(30),
    modelo varchar(30),
    ano int,
    primary key(id)
);

create table aluga(
     Data_init date,
    id_carro_fk int,
    numeroDias int,
    ativo boolean,
    finalizado boolean,
    inAberto boolean,
    id_cliente_fk int,
    foreign key (id_carro_fk) references carro(id),
    foreign key (id_cliente_fk) references cliente(id),
    primary key (Data_Init, id_carro_fk)
);
```

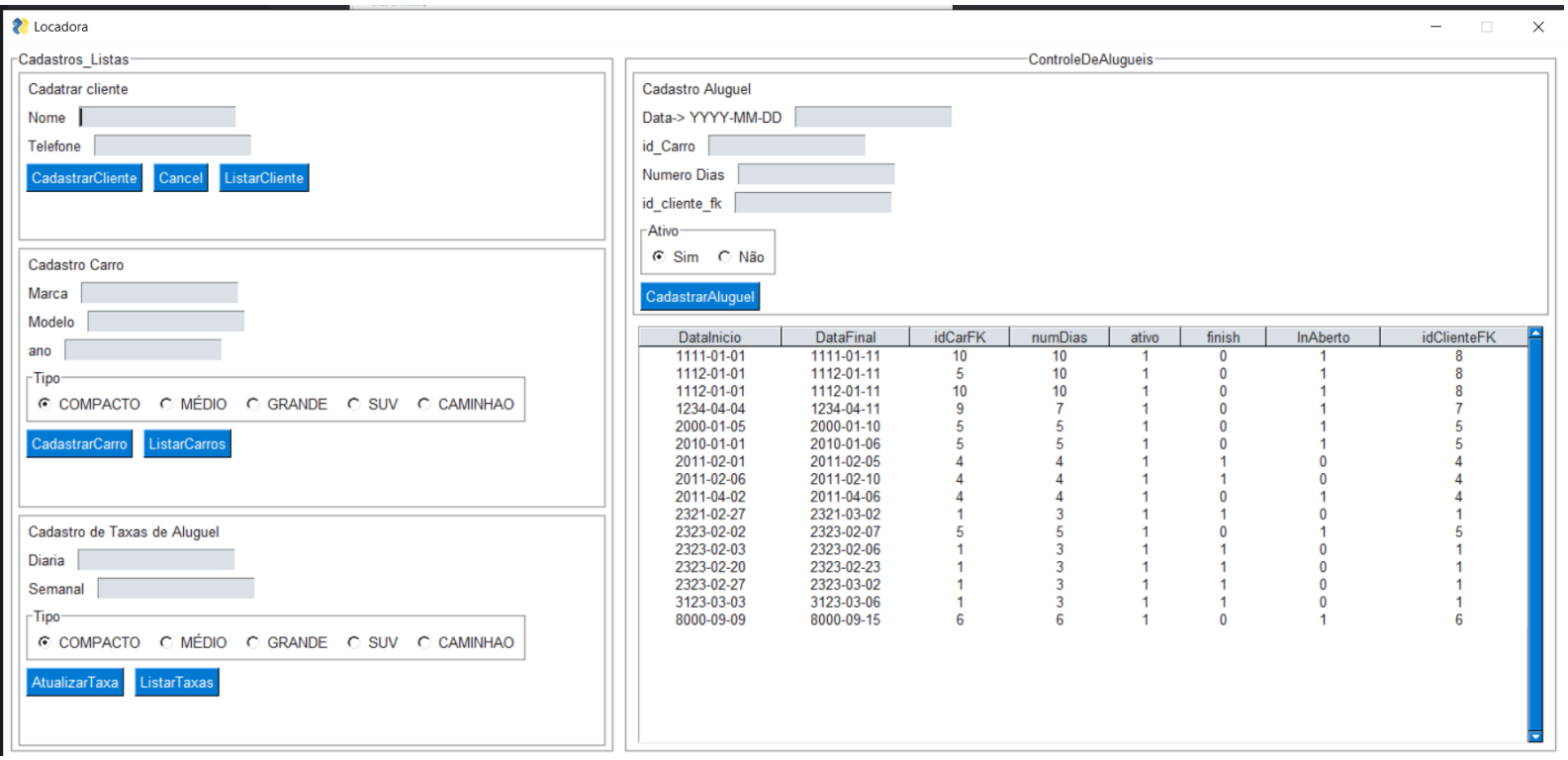
Agora os 10 inserts para **CLIENTE** e os 20 para **CARRO**.

```
INSERT INTO cliente(nome, telefone) VALUES ('Ana', 555555555);
INSERT INTO cliente(nome, telefone) VALUES ('Maria', 666666666);
INSERT INTO cliente(nome, telefone) VALUES ('Carlos', 777777777);
INSERT INTO cliente(nome, telefone) VALUES ('Luiza', 888888888);
INSERT INTO cliente(nome, telefone) VALUES ('Pedro', 999999999);
INSERT INTO cliente(nome, telefone) VALUES ('Mariana', 111111111);
INSERT INTO cliente(nome, telefone) VALUES ('Ricardo', 222222222);
INSERT INTO cliente(nome, telefone) VALUES ('Fernanda', 333333333);
INSERT INTO cliente(nome, telefone) VALUES ('Lucas', 444444444);
INSERT INTO cliente(nome, telefone) VALUES ('Isabela', 555555555);
```

```
INSERT INTO carro (tipo, diaria, semanal, marca, modelo, ano)
VALUES ('COMPACTO', 90, 530, 'Toyota', 'Corolla', 2018);
INSERT INTO carro (tipo, diaria, semanal, marca, modelo, ano)
VALUES ('MÉDIO', 120, 740, 'Ford', 'Cam', 2015);
INSERT INTO carro (tipo, diaria, semanal, marca, modelo, ano)
VALUES ('GRANDE', 320, 2.140, 'Honda', 'Accord', 2017);
INSERT INTO carro (tipo, diaria, semanal, marca, modelo, ano)
VALUES ('SUV', 116, 712, 'Chevrolet', 'Equinox', 2019);
INSERT INTO carro (tipo, diaria, semanal, marca, modelo, ano)
VALUES ('CAMINHÃO', 650, 4.450, 'Volvo', 'FH16', 2020);
INSERT INTO carro (tipo, diaria, semanal, marca, modelo, ano)
VALUES ('COMPACTO', 90, 530, 'Volkswagen', 'Golf', 2016);
INSERT INTO carro (tipo, diaria, semanal, marca, modelo, ano)
VALUES ('MÉDIO', 120, 740, 'Toyota', 'Camry', 2014);
INSERT INTO carro (tipo, diaria, semanal, marca, modelo, ano)
VALUES ('GRANDE', 320, 2.140, 'Honda', 'Civic', 2019);
INSERT INTO carro (tipo, diaria, semanal, marca, modelo, ano)
VALUES ('SUV', 116, 712, 'Jeep', 'Grand Cherokee', 2017);
INSERT INTO carro (tipo, diaria, semanal, marca, modelo, ano)
VALUES ('CAMINHÃO', 650, 4.450, 'Mercedes-Benz', 'Actros', 2018);
INSERT INTO carro (tipo, diaria, semanal, marca, modelo, ano)
VALUES ('COMPACTO', 90, 530, 'Renault', 'Clio', 2017);
INSERT INTO carro (tipo, diaria, semanal, marca, modelo, ano)
VALUES ('MÉDIO', 120, 740, 'Nissan', 'Sentra', 2016);
INSERT INTO carro (tipo, diaria, semanal, marca, modelo, ano)
VALUES ('GRANDE', 320, 2.140, 'Hyundai', 'Sonata', 2019);
INSERT INTO carro (tipo, diaria, semanal, marca, modelo, ano)
VALUES ('SUV', 116, 712, 'Subaru', 'Outback', 2018);
INSERT INTO carro (tipo, diaria, semanal, marca, modelo, ano)
VALUES ('CAMINHÃO', 650, 4.450, 'MAN', 'TGX', 2020);
INSERT INTO carro (tipo, diaria, semanal, marca, modelo, ano)
VALUES ('COMPACTO', 90, 530, 'Ford', 'Fiesta', 2017);
INSERT INTO carro (tipo, diaria, semanal, marca, modelo, ano)
VALUES ('MÉDIO', 120, 740, 'Chevrolet', 'Cruze', 2015);
INSERT INTO carro (tipo, diaria, semanal, marca, modelo, ano)
VALUES ('GRANDE', 320, 2.140, 'Audi', 'A6', 2019);
INSERT INTO carro (tipo, diaria, semanal, marca, modelo, ano)
VALUES ('SUV', 116, 712, 'Mazda', 'CX-5', 2018);
INSERT INTO carro (tipo, diaria, semanal, marca, modelo, ano)
VALUES ('CAMINHÃO', 650, 4.450, 'Scania', 'R730', 2021);
```

BANCO DE DADOS COM APLICAÇÃO:

Usamos como base a linguagem PYTHON e a API “PySimpleGUI” para a interface. Agora para a explicação da interface.



Essa é a interface inicial, nela possui, à esquerda, os Cadastros tirando o aluguel. Você pode **cadastrar o cliente**, escolhendo o nome em char, e número de telefone em bigint.

Cadastrar carro, escreva o nome da marca, modelo, e o ano. Além de você poder escolher nos radiobutton o tipo de carro que vai ser cadastrado assim como requisitado no trabalho.

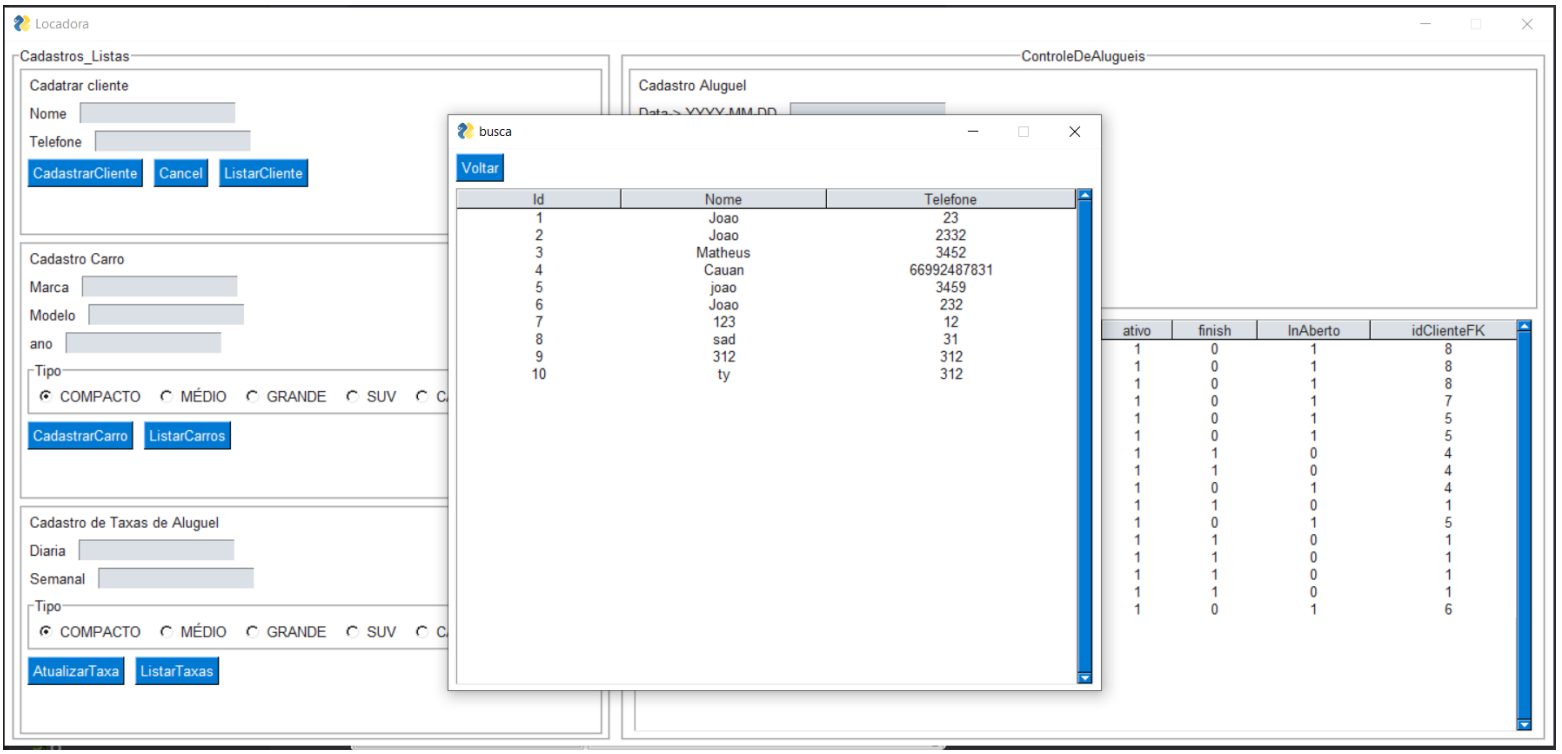
Cadastro de taxas de aluguel. Essa parte é interessante, pois, o valores das taxas possuem um padrão que logo irei mostrar, todavia, você pode alterar digitando nos campos de diária e semanal, lembrando que a opção selecionada no radiobutton é onde o valor vai ser aplicada.

A esquerda temos **Cadastrar Aluguel**. Aqui você pode cadastrar um aluguel com base nos CARROS cadastrados no BD, e para quem está com base no CLIENTE. Além de escolher o número de dias. Importante que em “Data” o formato pode ser **YYYY-M-D**. E escolher se vai estar ATIVO com sim ou não.

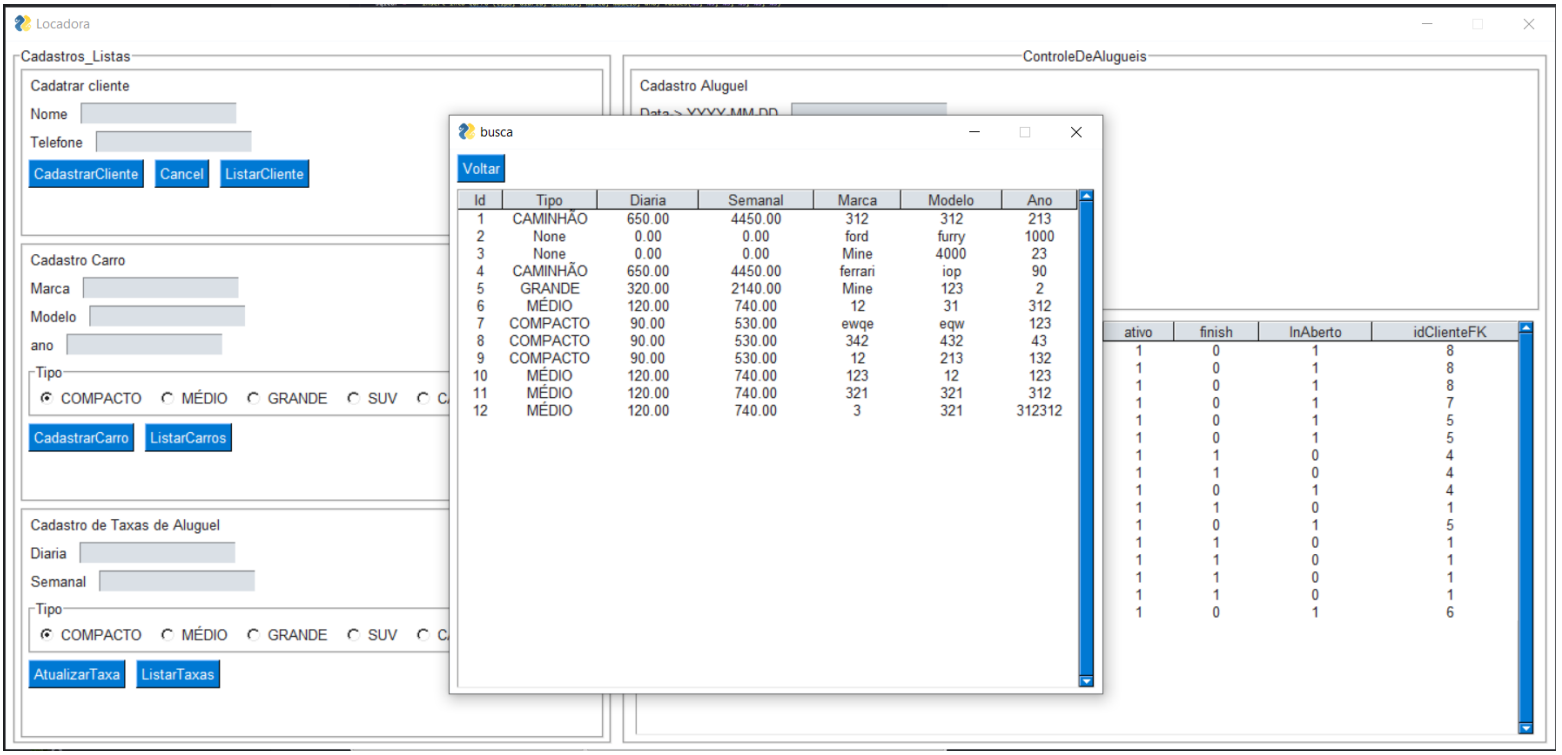
A última interface abaixo se refere aos aluguéis que você faz. Sendo que **DataFinal** é um cálculo de dias que você alugou.

Sendo de INICIO ao FIM das datas, ID do carro, números de dias para referência, se esta ATIVO, se esta FINALIZADO e ABERTO, além de mostrar o ID do cliente.

Agora seguindo de novo para as telas iniciais, vamos ver os “Listar”.
Seguindo a ordem **CLIENTE**, **CARRO**.

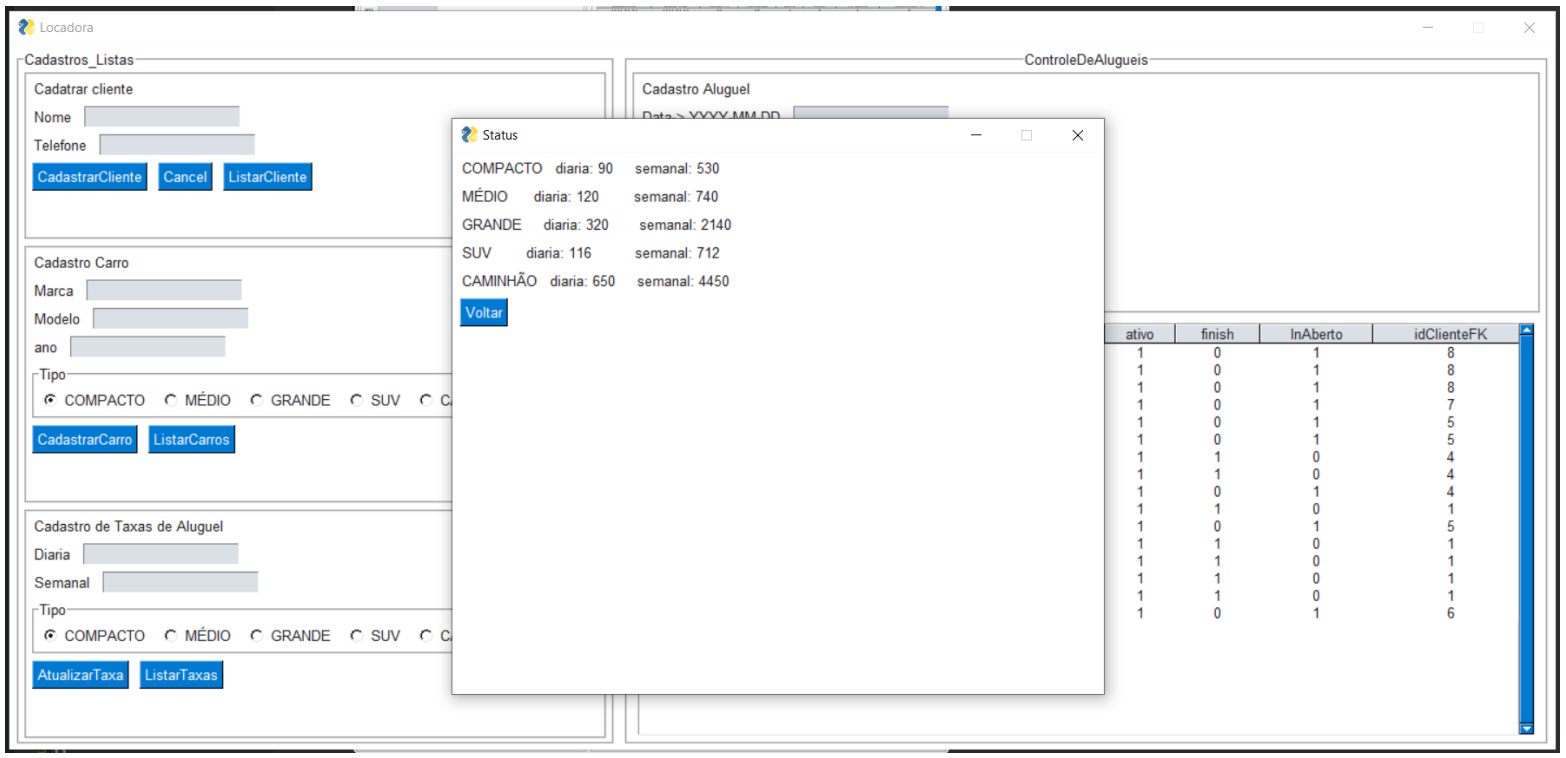


Ao clicar em Listar aparece esse window mostrando os CLIENTES cadastrados.



Em listar CARRO temos algo interessante, e umas informações extras. Aqui além de mostrar o CARRO cadastrado, mostra também já incluso, o valor calculado da DIÁRIA e SEMANAL, além das outras colunas já esperadas.

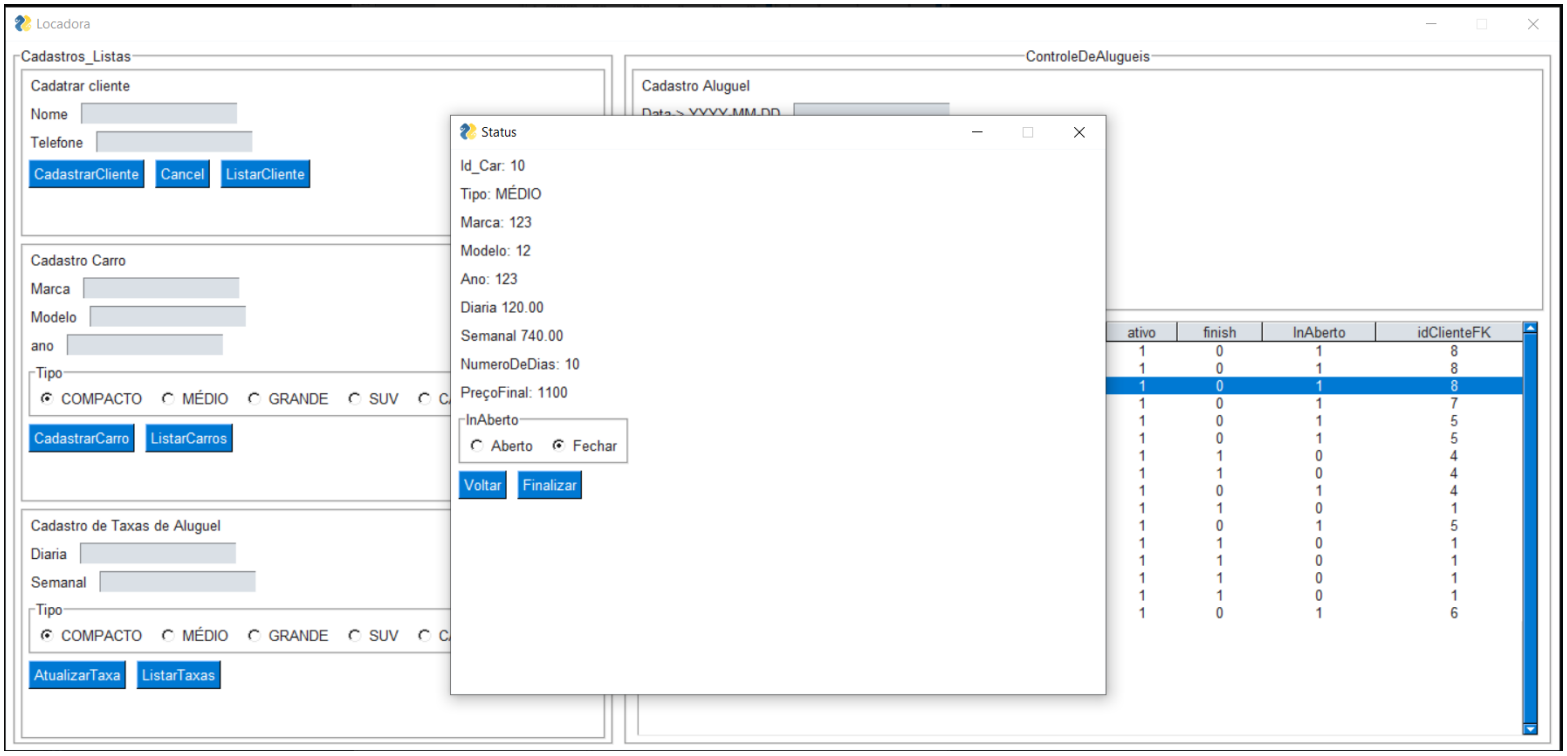
Tudo vai fazer sentido na tela de LISTAR TAXAS:



Essa é a tela de **ListarTaxas**, ela mostrar o geral de todos o TIPOS e seus valores, além disso mostra o cálculo semanal baseado na diária: (DIÁRIA * 7) e o desconto da semana: (DIÁRIA * 7) - DESCONTO.
Como visto no exemplo, a Diária de um COMPACTO custa 90, em sete dias é 630, e o desconto da semanal é 100, que dá 530.

IMPORTANTE: *Os valores agora alterados nas taxas só serão aplicados aos novos cadastros de CARRO, ou seja, não irá alterar todos os cadastros anteriores.*

Agora na última tela, ela funciona de forma especial e deve ser explicado previamente.



Voltando para a interface de aluguéis cadastrados, diferente dos outros, ele já está na amostra por ser uma tela de alta prioridade, todavia ela possui uma opção extra!
Ao clicar rápido duas vezes em uma TUPLA de um dos alugueis cadastrados, vai aparecer uma window que mostra o atual **STATUS** do aluguel, ou seja, nele você vai ver as informações do CARRO, junto dos valores das TAXAS, e o cálculo do número de dias.

Exemplo: Diaria 120 e o desconto semanal é 100, ou seja, 840 - 100 = 740. Como se passou apenas uma semana, esse desconto vai ser aplicado apenas uma vez. Nisso dando 1.100.

Além disso, aqui no STATUS você pode mudar de aberto para fechado e também FINALIZAR o pagamento, que, ao clicar em “Finalizar” você agora muda o “finish” da tabela de **ALUGUEL** de 0 para 1.

Nível De Isolamento: Repeatable Read

Decidi escolher para esse programa o nível de isolamento repeatable read, pois achei ele o mais ideal para o propósito de negócio de vendas, pois ele vai proteger os dados lidos que estiverem sendo processados de outras transações o que vai evitar inúmeros problemas de segurança que são inaceitáveis para um programa de venda permitir que ocorram, além de que é uma opção mais eficiente em desempenho do que o serializable, pois ela não vai interferir nas transações que não precisam acessar dados não usados além de outros fatores, e também apesar de menor em desempenho do que o read committed, ela trará mais segurança como já discutido, o que faz ser um bom equilíbrio entre segurança e desempenho. Ela já é o nível de isolamento padrão do mysql.

Transações:

Como o MySql-Connector Inicia a conexão por padrão no modo transacional, não há necessidade de iniciá la de maneira explícita, caso qualquer erro ocorra durante a execução do código, qualquer que seja, essa exceção será capturada e automaticamente será feito rollback() através da variável con(connector), e todas as alterações serão desmanchadas para assim a integridade do programa seja mantida em segurança e respeitando sua atomicidade.

```
    except Exception as e:
        con.rollback()
        print(e)
        pyGui.popup(f'[Error]{e}')
        return 1
```

Triggers e Procedimentos:

Foram feitos 2 triggers e 3 procedimentos, todos usados para o propósito de verificação de integridade do banco:

```
#-----------------------------------------------aluga-----------------------------------------
DELIMITER $
CREATE TRIGGER VerificaAlugaCarro BEFORE insert
ON aluga
FOR EACH ROW
BEGIN
    call VerificaProcAlugaCarro(NEW.id_carro_fk);

END$
DELIMITER ;
```

Esse 1 trigger “VerificaAlugaCarro” é disparado antes de 1 inserção na tabela aluga e

chama procedimento “VerificProcAlugaCarro”.

```
#----------------------------------------------aluga----------------------------------------
DELIMITER $
CREATE PROCEDURE VerificaProcAlugaCarro (in auxID int)
BEGIN
    declare a int;
    select count(id) into a from carro where carro.id = auxID;
    if a = 0 then
        signal SQLSTATE '45000'
            set message_text = 'Esse Carro não Existe No Banco de Dados';
    end if;

END $
DELIMITER ;
```

Esse procedimento vai verificar se dentro do banco o id do carro passado existe, juntando os 2 seu propósito único é evitar que carros que não existem no banco sejam inseridos em tuplas dentro da tabela aluga, e caso um carro inexistente esteja sendo passado, o mysql emitirá um erro que será capturado dentro do try/catch do python que tratará do ocorrido.

Semelhante ao trigger e procedimento anterior, esses em conjunto farão a mesma verificação só que agora verificando o id do cliente para a tabela aluga:

```
#----------------------------------------------cliente----------------------------------------
DELIMITER $
CREATE TRIGGER VerificaAlugaCliente BEFORE insert
ON aluga
FOR EACH ROW
BEGIN
    call VerificaProcAlugaCliente(NEW.id_cliente_fk);

END$
DELIMITER ;
```

```
#----------------------------------------------cliente----------------------------------------
DELIMITER $
CREATE PROCEDURE VerificaProcAlugaCliente (in auxID int)
BEGIN
    declare a int;
    select count(id) into a from cliente where cliente.id = auxID;
    if a = 0 then
        signal SQLSTATE '45000'
            set message_text = 'Esse Cliente não Existe No Banco de Dados';
    end if;

END $
DELIMITER ;
```

Agora analisando o 3º e último procedimento, chamado “VerificaProcTel”.

```
#-----------------------------------------------telefone-----------------------------------------------
DELIMITER $
CREATE PROCEDURE VerificaProcTel (in auxID bigint,  out ex bool)
BEGIN
    declare a bigint;
    select count(telefone) into a from cliente where telefone = auxID;
    if a = 0 then
    set ex = true;
    else
    set ex = false;
    end if;

END $
DELIMITER ;
```

Esse procedimento é chamado dentro do código python usando cursor() através da função callproc, e está sendo usado para verificar se no cadastro de um novo cliente no sistema, se seu número de telefone já não existe cadastrado no sistema, pois um número de telefone para aluguel de carros é um contato privado e não faz sentido seu número já estar com outra pessoa.

```
60                  args = cur.callproc('VerificaProcTel', [values['telC'], 0])
61                  if(args[1] == True):
62                      cur.execute(sql, (params)) #usando tupla
63                      con.commit()
64                      pyGui.popup('Cadastrado')
65                  else:
66                      pyGui.popup('ConsultaParam Descobriu que esse numero de celular ja foi cadastrado')
```

Dentro do procedimento será feito a contagem de quantos telefones forem iguais ao que está sendo passado, e dependendo do que for retornado a variável de saída “ex” do procedimento que receberá no final verdadeiro ou falso, nos então pegamos o que foi retornado agora dentro de python através da função callproc() e continuamos o programa de acordo com o resultado.